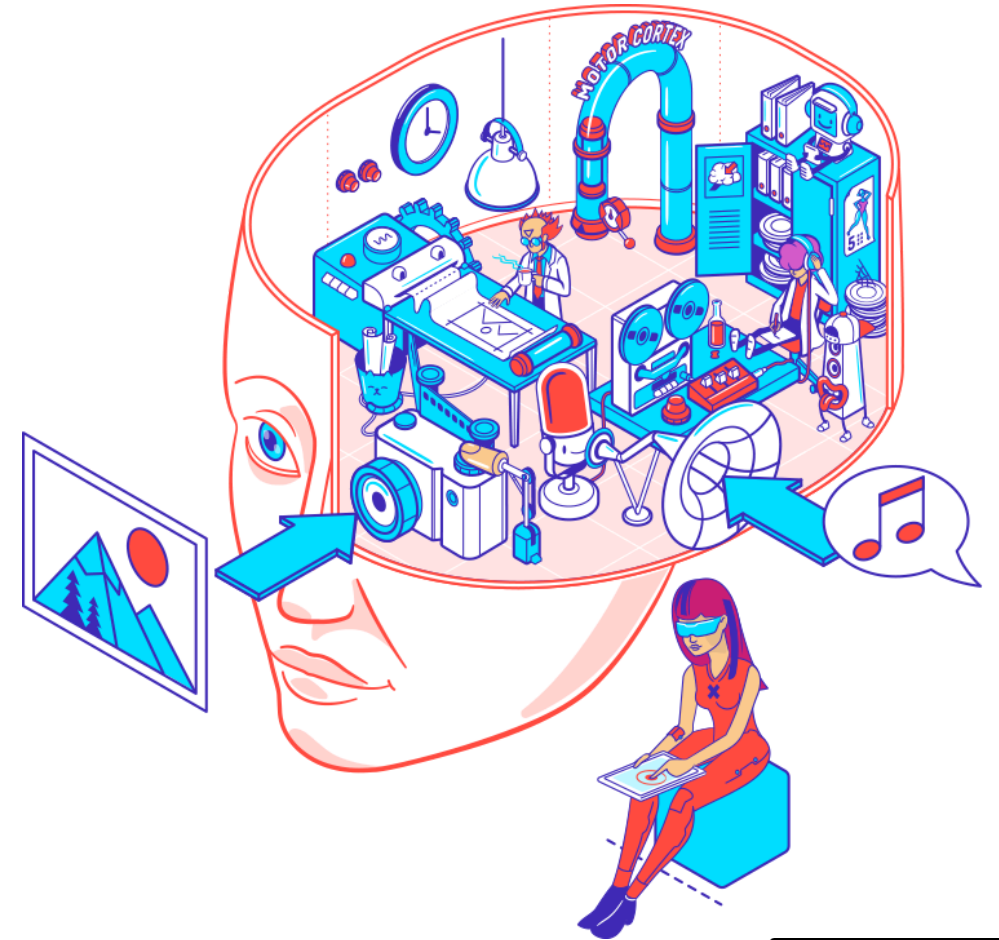


MEDIA PROGRAMMING

UNIT 3: RASTER GRAPHICS

Prof. Dr. Eike Langbehn

Department of Media Technology
Faculty of Design, Media and Information
Hamburg University of Applied Sciences



AGENDA:

MEDIA PROGRAMMING

UNIT 3: RASTER GRAPHICS

INTRO

1. Introduction
2. From Java to Python
3. Raster Graphics
4. Algorithmic Graphics Generation
5. Vector Graphics
6. Interactive Graphics
7. Animations

LEARNING OUTCOME

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

INTRO

- Understanding **basics of raster graphics**
- Knowledge about **encoding & compression** of raster graphics
- Knowing how to apply simple **filters**

BASICS

UNIT 3: RASTER GRAPHICS



RASTER GRAPHICS

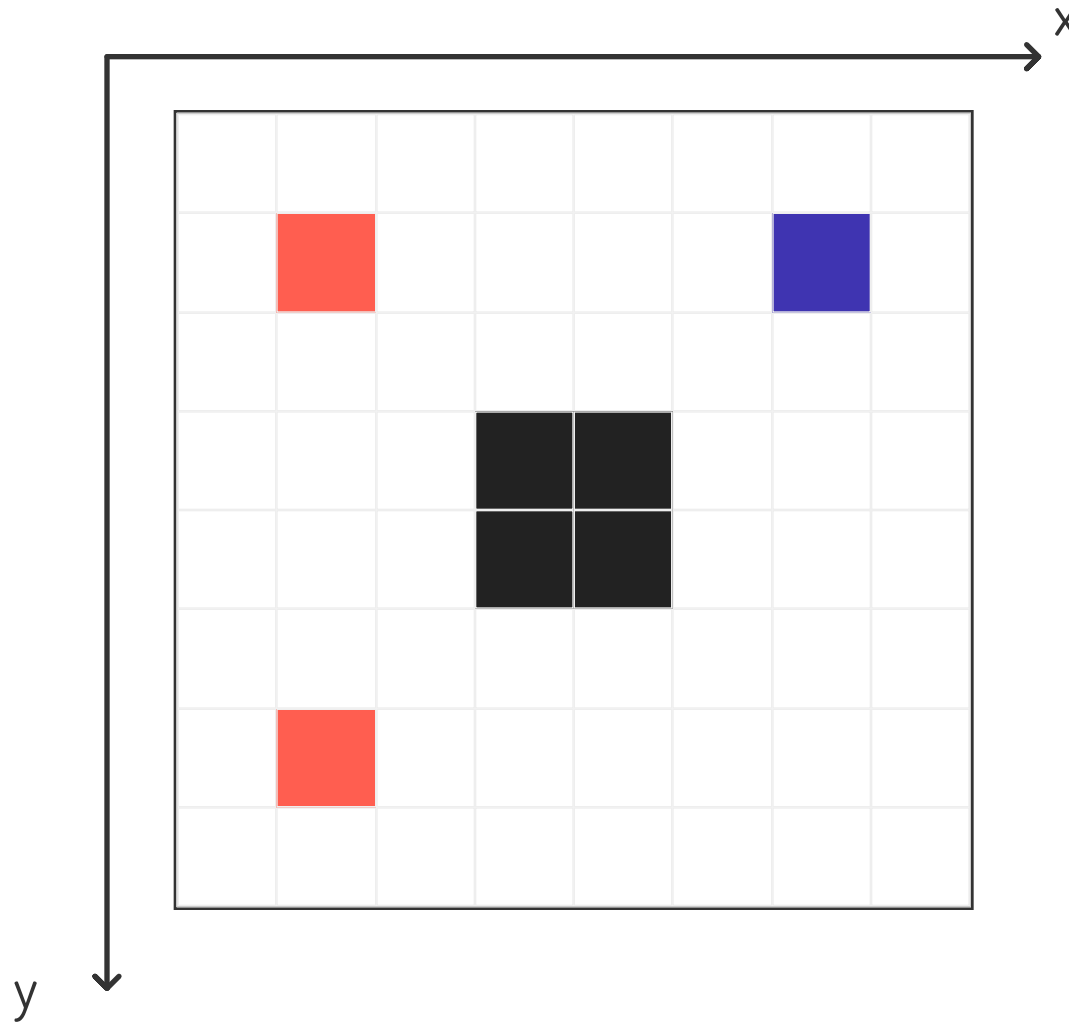
- **Definition:** Digital images created from a rectangular grid of pixels (picture elements).
- **Pixel Data:** Each pixel contains specific color information (RGB/RGBA).
- **Complexity:** Ideal for photographic images with subtle gradients and complex details.
- **Storage:** File size is determined by resolution and bit depth (bits per pixel).

ANATOMY OF A RASTER GRID

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

SECTION



- **Origin (0,0):** Usually top-left.
- **Pixels:** Discrete grid cells.
- **Addresses:** Each pixel is identified by (x, y).

ANATOMY OF A RASTER GRID

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

SECTION

```
1 # A simple 2D list representing a grayscale image
2 # 0 = Black, 255 = White
3 raster_image = [
4     [0, 0, 0, 0, 0],
5     [0, 255, 0, 255, 0],
6     [0, 0, 0, 0, 0],
7     [0, 255, 255, 255, 0],
8     [0, 0, 0, 0, 0]
9 ]
10
11 # Accessing a specific pixel (x=1, y=3)
12 pixel_value = raster_image[3][1]
13 print(f"Pixel at (1,3): {pixel_value}")
```

MAGNIFICATION

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

BASICS



ZOOM



Pixelation: Scaling up revealed the discrete squares (pixels). Information cannot be "created" by zooming.

RASTER FILE FORMATS

Format	Compression	Transparency	Use Case
JPEG	Lossy	No	Photos, complex color gradients
PNG	Lossless	Yes (Alpha)	Web graphics, icons, logos
GIF	Lossless (LZW)	Binary (1-bit)	Simple animations, limited colors (256)
WebP	Lossy/Lossless	Yes	Modern web optimization
TIFF	Usually Lossless	Yes	High-quality printing, archiving

RASTER GRAPHICS SOFTWARE

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

BASICS

PROFESSIONAL / PAID

- **Adobe Photoshop:** The industry standard for photo editing and digital painting.
- **Affinity Photo:** A powerful, professional alternative.
- **Corel Painter:** Specialized in mimicking natural media (oils, charcoal, etc.).

FREE / OPEN SOURCE

- **GIMP:** Powerful open-source alternative to Photoshop.
- **Krita:** Excellent for digital painting and 2D animation.
- **Paint.NET:** Simple yet capable editor for Windows.
- **Pixlr / Photopea:** Robust browser-based editors.

ADVANTAGES

- **Detail:** Capable of displaying complex, photographic images with subtle color transitions.
- **Pixel-level Control:** Each pixel can be edited individually for precise manipulation.
- **Broad Compatibility:** Supported by almost all image viewers, browsers, and hardware.
- **Rich Toolsets:** Extensive libraries and software (Photoshop, GIMP) for advanced editing.

DISADVANTAGES

- **Scalability:** Significant loss of quality (pixelation) when enlarged.
- **File Size:** High-resolution images can be extremely large, requiring heavy compression.
- **Editing Constraints:** Difficult to isolate objects once they are merged into the grid.
- **Resolution Fixed:** Once captured, detail is capped by the initial sensor/render settings.

ENCODING

UNIT 3: RASTER GRAPHICS



RGB & RGBA COLOR MODELS

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

ENCODING

RGB MODEL

- **Additive Color:** Red, Green, and Blue combine to create light.
- **Components:** Usually 3 channels (8-bit each, 0-255).
- **Black:** (0, 0, 0) - All lights off.
- **White:** (255, 255, 255) - All lights at max intensity.

RGBA EXTENSION

- **Alpha Channel:** Controls the opacity/transparency of the pixel.
- **Values:** 0.0 (fully transparent) to 1.0 (fully opaque), or 0-255.
- **Compositing:** Allows for layering images and semi-transparent effects.

```
1 # RGBA Pixel Representation
2 red_full = (255, 0, 0, 255)    # Solid Red
3 red_half = (255, 0, 0, 128)    # Semi-transparent Red
4 invisible = (0, 0, 0, 0)       # Fully Transparent
```

RESOLUTION & COLOR DEPTH

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

ENCODING

RESOLUTION

The total number of pixels in an image, typically expressed as **Width × Height**.

- **Density:** Often measured in PPI (Pixels Per Inch).
- **Detail:** Higher resolution allows for more fine detail.
- **Impact:** Directly affects the uncompressed file size.

```
1 # Calculating the number of possible colors
2 bit_depth = 24
3 num_colors = 2**bit_depth
4 print(f"{num_colors:,} colors") # 16,777,216
```

COLOR DEPTH

Also known as **Bit Depth**; the number of bits used to represent the color of a single pixel.

- **1-bit:** 2 colors (Monochrome).
- **8-bit:** 256 colors (Indexed).
- **24-bit:** 16.7 million colors (True Color).
- **Higher depth (10-16 bit per channel):** Smoother gradients, less "banding", over 1 billion to 281 trillion colors (HDR).

MEMORY STORAGE

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

ENCODING

- Higher resolution means more detail but larger file sizes.
- $\text{Filesize} \approx \text{Resolution} \times \text{Color Depth} / 8$ (bytes).
- The image data is stored sequentially in memory
- Most formats use the convention that the pixel data is stored row-wise

STORAGE CALCULATION EXAMPLE

To calculate the uncompressed file size (or RAM usage) for a Full HD image with **24-bit** color:

- **Resolution:** $1920 \times 1080 = 2,073,600$ pixels
- **Total Bits:** $2,073,600 \times 24 = 49,766,400$ bits
- **Total Bytes:** $49,766,400 / 8 = 6,220,800$ bytes
- **Final Size:** ~5.93 MB

HOW IT'S STORED IN RAM

Raster data is stored as a **one-dimensional array** (buffer) in memory. The computer calculates the memory offset for a pixel at (x, y) using:

$$\text{Offset} = (y * \text{width} + x) * (\text{bits_per_pixel} / 8)$$

COMPRESSION

UNIT 3: RASTER GRAPHICS



JPEG COMPRESSION

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

COMPRESSION

JPEG is a **lossy** compression standard optimized for photographic images, utilizing the human eye's limitations.

PIPELINE

1. **Color Space Conversion:** RGB to YCbCr (separating brightness from color).
2. **Chroma Subsampling:** Reducing resolution of color channels.
3. **Block Splitting:** Dividing the image into 8×8 pixel blocks.
4. **DCT (Discrete Cosine Transform):** Converting spatial data to frequency data.
5. **Quantization:** Reducing precision of high frequencies (where the loss happens).
6. **Entropy Coding:** Final lossless compression (Huffman/Run-length).

RECAP: VISUAL PERCEPTION

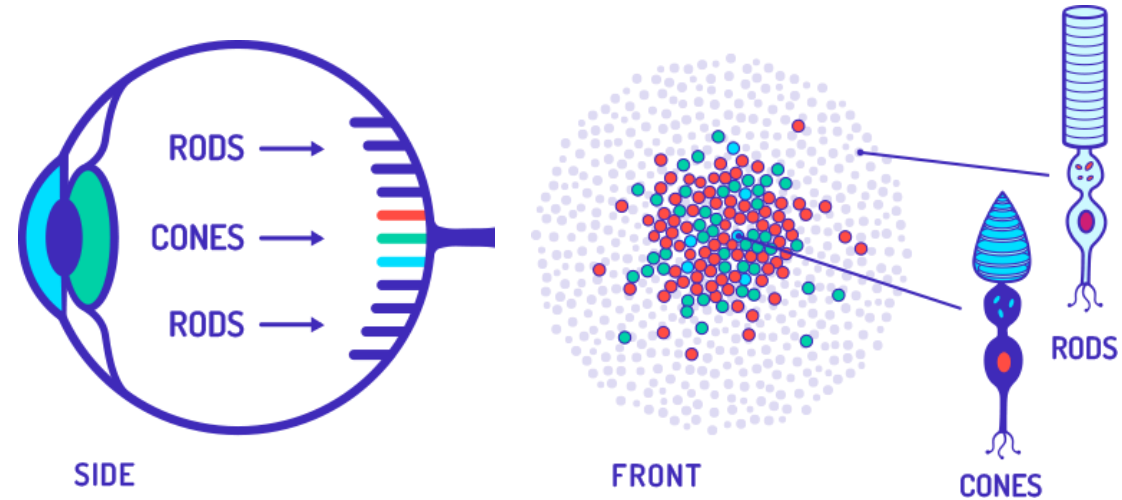
GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

COMPRESSION

Why it works

The human visual system is more sensitive to changes in brightness than in color, and less sensitive to high-frequency noise. JPEG exploits this by discarding details that we likely won't notice.



- Perception of color, shape, depth, movements etc.
- **RETINA** is innermost, light-sensitive layer of the eye
- Photoreceptive cells transform light into neuronal activities and forward these information
 - **CONES (7-8 MIO)**: colors & shape
 - **RODS (100-120 MIO)**: light/dark, motion

YCBCR COLOR SPACE

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

COMPRESSION

YCBCR COLOR MODEL

YCbCr is a color model used for "decoupling" brightness from color data.

- Y (Luma): Brightness channel
- Cb (Chroma Blue): Color channel
- Cr (Chroma Red): Color channel

$$Y = 0.299R + 0.587G + 0.114B$$

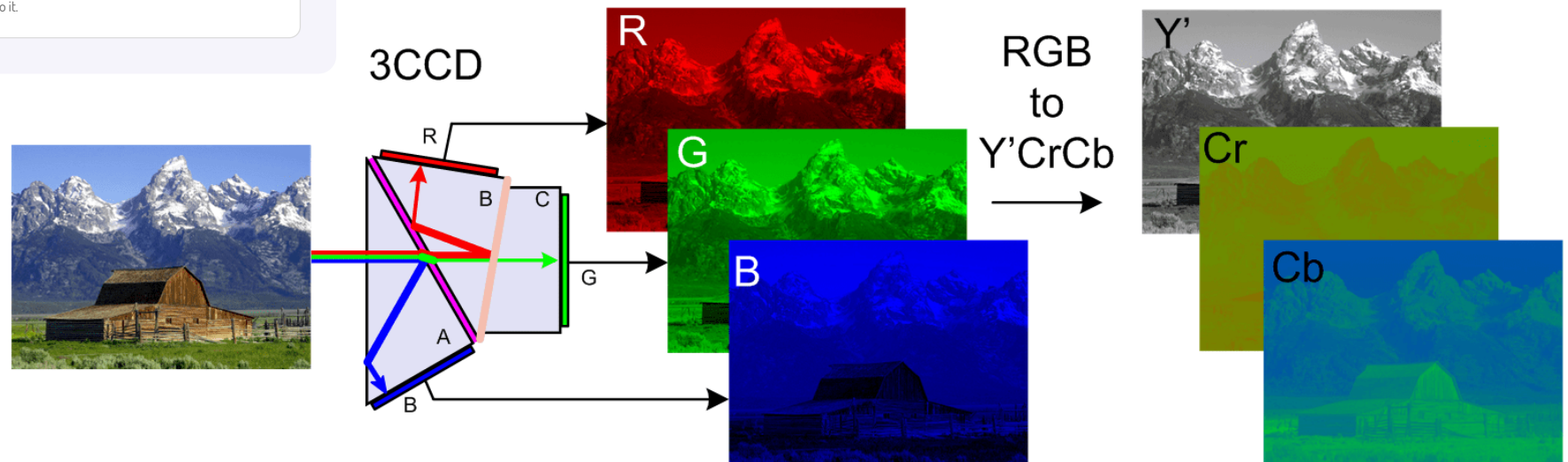
$$Cb = (B - Y) * 0.564$$

$$Cr = (R - Y) * 0.713$$

Note: Green has the highest weight because the human eye is most sensitive to it.

Why YCbCr for JPEG?

By separating brightness (Y) from color (Cb/Cr), we can compress the color channels more aggressively without the human eye noticing much difference, as our eyes are far more sensitive to brightness detail.



CHROMA SUBSAMPLING

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

COMPRESSION

THE CONCEPT OF SUBSAMPLING

Chroma subsampling is the practice of encoding images by implementing less resolution for chroma (color) information than for luma (brightness).

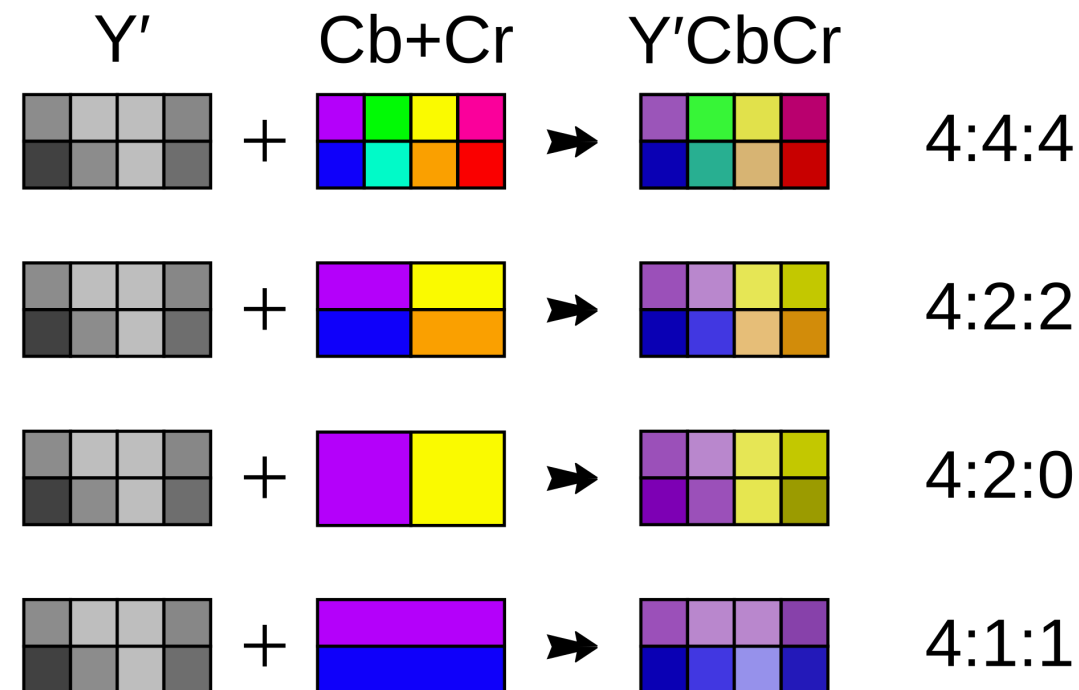
- **4:4:4** - No subsampling (Original quality).
- **4:2:2** - Half the horizontal chroma resolution.
- **4:2:0** - Quarter the chroma resolution (Common in JPEG).

Size Reduction

Subsampling at **4:2:0** reduces the amount of data needed for color by 75%. This results in a total uncompressed file size reduction of **50%** (from 24 bits per pixel to 12 bits per pixel) before any further compression takes place.

Mechanism

In a 2x2 pixel block, 4:2:0 subsampling takes the average color of all four pixels and uses it for the entire block, while keeping unique brightness values for every single pixel. This averages 4 chroma samples into 1.



DISCRETE COSINE TRANSFORM

GRAPHICS PROGRAMMING

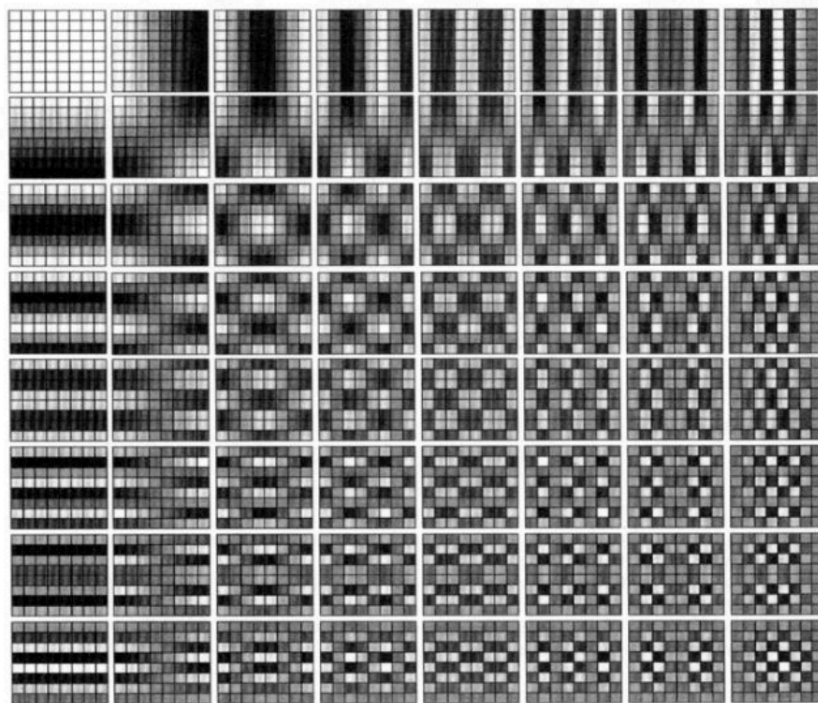
UNIT 3: RASTER GRAPHICS

COMPRESSION

WHAT IS DCT?

The Discrete Cosine Transform (DCT) is a mathematical process that transforms image data from the **spatial domain** (pixels) to the **frequency domain**.

- **Input:** 8×8 blocks of pixel values (brightness/color).
- **Output:** 8×8 matrix of coefficients representing frequencies.

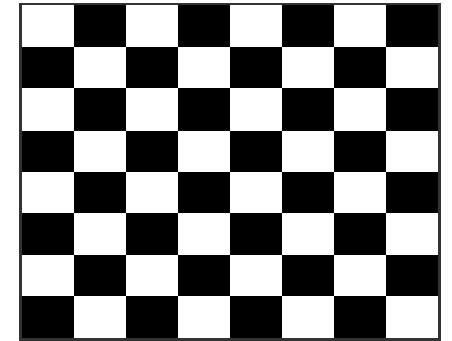


Low Frequency Example



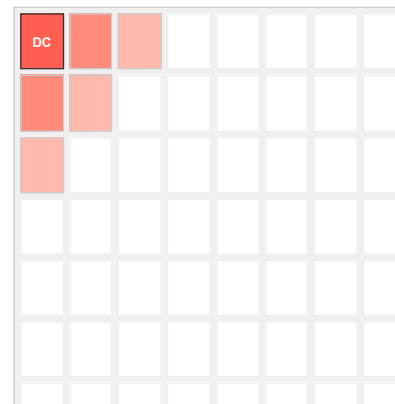
Slow changes = Low Frequency

High Frequency Example



Rapid changes = High Frequency

8x8 Coefficient Matrix



■ Low Freq (DC) ■ High Freq

1. Most of the important visual information is concentrated in the top-left (low frequency) coefficients.
2. This step is **lossless** by itself; the loss happens during the subsequent Quantization step.

QUANTIZATION

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

COMPRESSION

LOSSY COMPRESSION STEP

Quantization is the primary **lossy** stage in JPEG compression. It reduces the precision of the DCT coefficients by dividing them by a quantization matrix and rounding to the nearest integer.

- **High Frequencies:** Heavily rounded (often to zero) as the human eye is less sensitive to fine detail.
- **Low Frequencies:** Preserved with higher precision to maintain overall image structure.
- **Result:** Drastic reduction in data size at the cost of slight visual fidelity.

Before: High Precision

123.456...



After: Rounded/Simplified

120

Rounding discards details that are hard for the human eye to see.

$$B_{i,j} = \text{round} \left(\frac{DCT_{i,j}}{Q_{i,j}} \right)$$

ENTROPY CODING

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

COMPRESSION

PACKING THE DATA

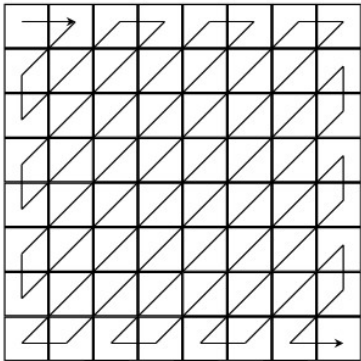
Entropy coding is the final, **lossless** stage of JPEG compression. It converts the quantized coefficients into a highly efficient binary stream.

- **Zig-Zag Scan:** Coefficients are reordered in a zig-zag pattern to group long runs of zeros together.
- **RLE (Run-Length Encoding):** Efficiently stores those consecutive zeros.
- **Huffman Coding:** Assigns shorter binary codes to frequently occurring values.

```
// Quantized Block (Top-Left)
[ 26, -3, 1, 0, 0, 0, ... ]
```



```
// Binary Stream (Result)
1101001011000101110...
```



RUN-LENGTH ENCODING (RLE)

Input:

0, 0, 0, 0, 0, 0, 4, 0, 0



Compressed:

(6, 0), (1, 4), (2, 0)

Stores "runs" of identical values as (count, value).



HUFFMAN CODING

'0' → 1

'4' → 01

'26' → 001

Most frequent values
get the **shortest** binary
codes.

PROCESSING

UNIT 3: RASTER GRAPHICS



IMAGE PROCESSING

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

PROCESSING

Point Ops:

- Changes pixels individually
- Adjusting e.g. brightness or contrast

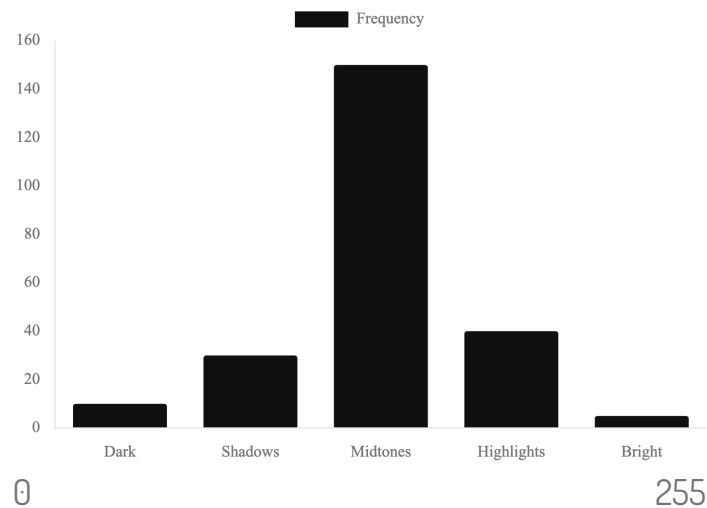
Filters:

- Changes pixels based on their neighbourhood
- E.g. smoothing (blur) or sharpening (edge detection)

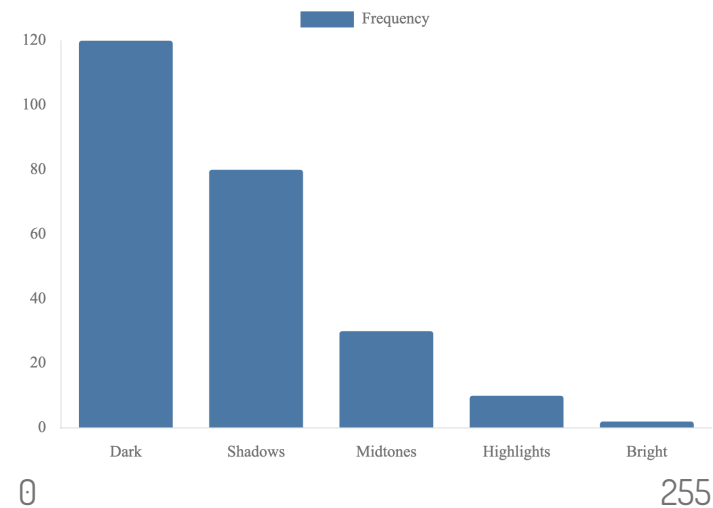
HISTOGRAMS

A **Histogram** is a graphical representation of the distribution of numerical data. In digital imaging, it displays the **frequency** of each intensity value (0-255) present in an image.

- **X-axis:** Represents the tonal range (Black on the left, White on the right).
- **Y-axis:** Represents the number of pixels at each specific tonal level.



Well Exposed



Underexposed

POINT OPS: BINARIZATION

GRAPHICS PROGRAMMING

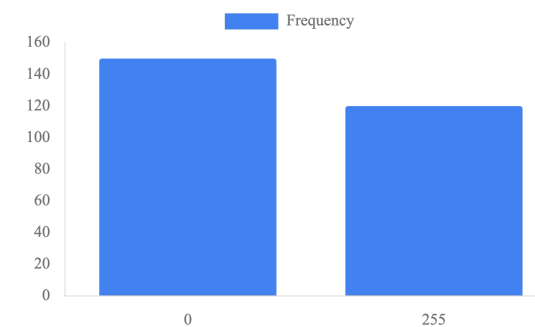
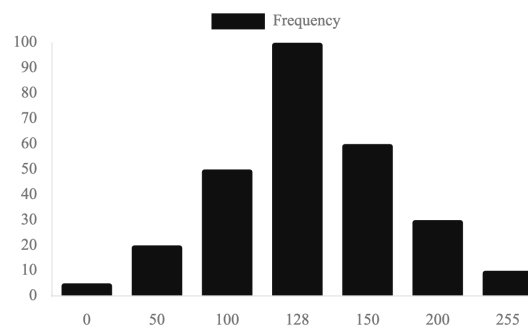
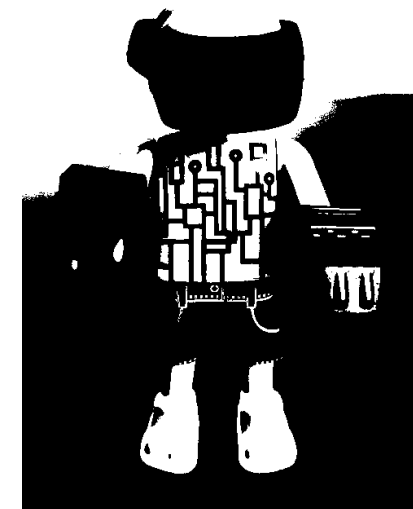
UNIT 3: RASTER GRAPHICS

PROCESSING



$$f'(x, y) = \begin{cases} 0 & \text{if } f(x, y) < T \\ 255 & \text{if } f(x, y) \geq T \end{cases}$$

$T = 128$



POINT OPS: NORMALIZATION

GRAPHICS PROGRAMMING

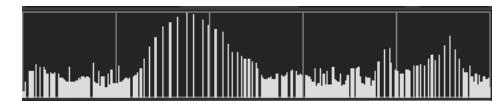
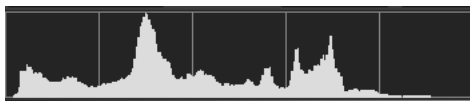
UNIT 3: RASTER GRAPHICS

PROCESSING



$$f'(x, y) = \frac{f(x, y) - h_{min}}{h_{max} - h_{min}} \times w_{max}$$

STRETCH



FILTERS: BLUR

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

PROCESSING

BOX BLUR (3×3)

The simplest blur filter calculates the average of a pixel and its 8 neighbors. Each pixel in the result is the sum of the region divided by 9.

$$f'(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 f(x + i, y + j)$$



BLUR



$$f'(x, y) = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} f(x-1, y-1) & f(x, y-1) & f(x+1, y-1) \\ f(x-1, y) & f(x, y) & f(x+1, y) \\ f(x-1, y+1) & f(x, y+1) & f(x+1, y+1) \end{bmatrix}$$

FILTERS: BOOST

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

PROCESSING

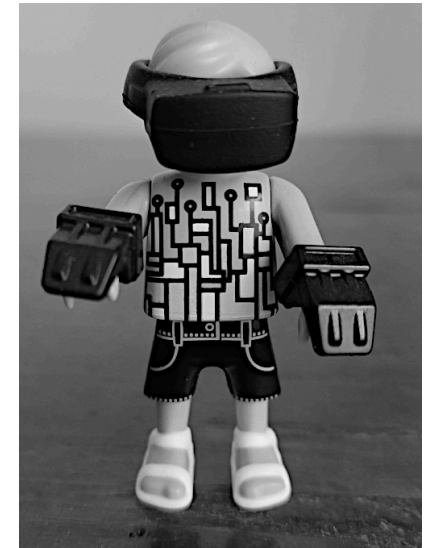
BOOST FILTER (3×3)

The boost filter amplifies details by emphasizing the center pixel and subtracting a portion of its neighbors. This sharpens edges and enhances high-frequency content.

$$f'(x, y) = 9 \cdot f(x, y) - \sum_{(i,j) \neq (0,0)} f(x + i, y + j)$$



BOOST
→



$$f'(x, y) = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix} \times \begin{bmatrix} f(x-1, y-1) & f(x, y-1) & f(x+1, y-1) \\ f(x-1, y) & f(x, y) & f(x+1, y) \\ f(x-1, y+1) & f(x, y+1) & f(x+1, y+1) \end{bmatrix}$$

FILTERS: EDGE DETECTION

GRAPHICS PROGRAMMING

UNIT 3: RASTER GRAPHICS

PROCESSING

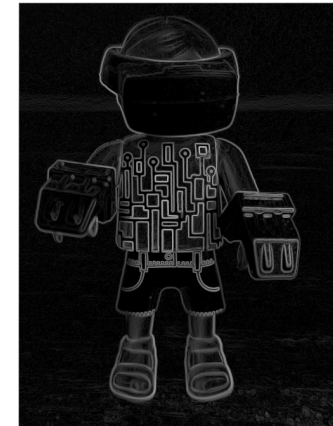
SOBEL OPERATOR

Used for edge detection by calculating the gradient of image intensity. It emphasizes high spatial frequency regions that correspond to edges.

$$f'(x, y) = \sqrt{\left(\sum_{i,j} K_x \cdot f_{i,j}\right)^2 + \left(\sum_{i,j} K_y \cdot f_{i,j}\right)^2}$$



SOBEL



$$G = \sqrt{G_x^2 + G_y^2} \quad G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * A, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * A$$

END

UNIT 3: RASTER GRAPHICS



Unless otherwise noted, the contents of this Open Educational Resource are licensed under the Creative-Commons Attribution 4.0 International license (CC BY 4.0)

Attribution: Eike Langbehn, Anh Sang Tran, Peter Wood

